

Undoing a Move

Next, as part of the backtracking in the recursive algorithm, our program needs to undo moves. This is as simple as making a move in the opposite direction as the initial move. The Python code for the `undoMove()` function is as follows:

Python

```
def undoMove(board, move):
    """Do the opposite move of `move` to undo it on `board`."""
    if move == UP:
        makeMove(board, DOWN)
    elif move == DOWN:
        makeMove(board, UP)
    elif move == LEFT:
        makeMove(board, RIGHT)
    elif move == RIGHT:
        makeMove(board, LEFT)
```

The JavaScript code for the `undoMove()` function is as follows:

JavaScript

```
function undoMove(board, move) {
    // Do the opposite move of `move` to undo it on `board`.
    if (move === UP) {
        makeMove(board, DOWN);
    } else if (move === DOWN) {
        makeMove(board, UP);
    } else if (move === LEFT) {
        makeMove(board, RIGHT);
    } else if (move === RIGHT) {
        makeMove(board, LEFT);
    }
}
```

We've already programmed the swapping logic into the `makeMove()` function, so `undoMove()` can just call that function for the direction opposite of the `move` argument. This way, a hypothetical `someMove` move made on a hypothetical `someBoard` data structure by the `makeMove(someBoard, someMove)` function call can be undone by calling `undoMove(someBoard, someMove)`.